

Nested vectored interrupt controller (NVIC)

Features

- 68 (not including the sixteen Cortex[®]-M3 interrupt lines)
- 16 programmable priority levels (4 bits of interrupt priority are used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of System Control Registers

The NVIC and the processor core interface are closely coupled, which enables low latency interrupt processing and efficient processing of late arriving interrupts.

All interrupts including the core exceptions are managed by the NVIC. For more information on exceptions and NVIC programming, refer to *STM32F10xxx Cortex[®]-M3 programming manual* (see [Related documents on page 1](#)).

Table 63. Vector table for other STM32F10xxx devices

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000_0000
-	-3	fixed	Reset	Reset	0x0000_0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
-	-1	fixed	HardFault	All class of fault	0x0000_000C
-	0	settable	MemManage	Memory management	0x0000_0010
-	1	settable	BusFault	Prefetch fault, memory access fault	0x0000_0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
-	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000_0030
-	-	-	-	Reserved	0x0000_0034
-	5	settable	PendSV	Pendable request for system service	0x0000_0038
-	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
3	10	settable	RTC	RTC global interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
11	18	settable	DMA1_Channel1	DMA1 Channel1 global interrupt	0x0000_006C
12	19	settable	DMA1_Channel2	DMA1 Channel2 global interrupt	0x0000_0070
13	20	settable	DMA1_Channel3	DMA1 Channel3 global interrupt	0x0000_0074



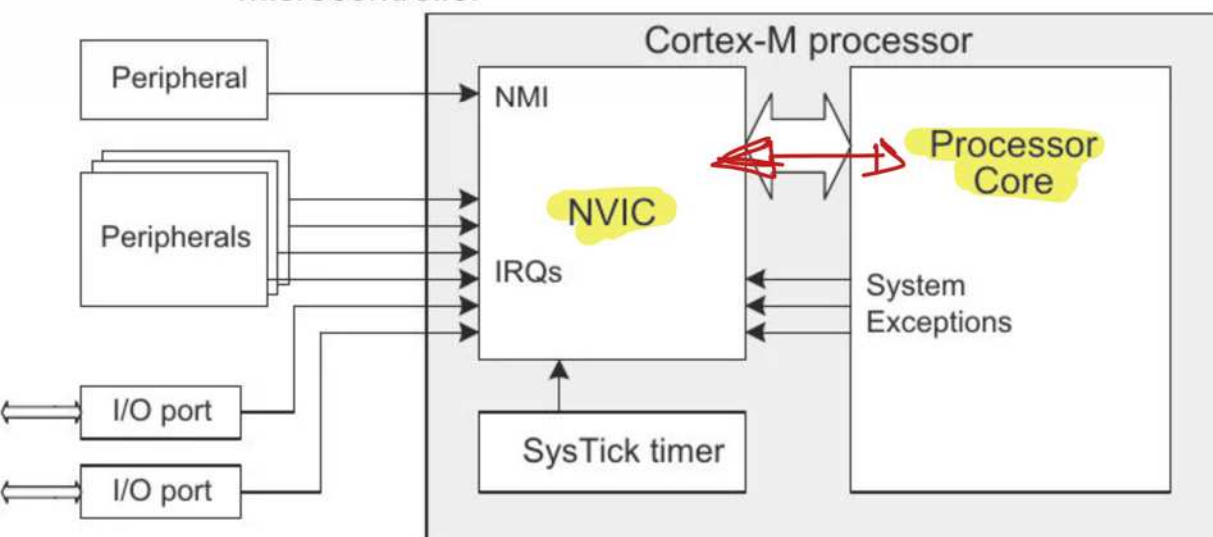
Table 63. Vector table for other STM32F10xxx devices (continued)

Position	Priority	Type of priority	Acronym	Description	Address
14	21	settable	DMA1_Channel4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA1_Channel5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA1_Channel6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA1_Channel7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1_2	ADC1 and ADC2 global interrupt	0x0000_0088
19	26	settable	USB_HP_CAN_TX	USB High Priority or CAN TX interrupts	0x0000_008C
20	27	settable	USB_LP_CAN_RX0	USB Low Priority or CAN RX0 interrupts	0x0000_0090
21	28	settable	CAN_RX1	CAN RX1 interrupt	0x0000_0094
22	29	settable	CAN_SCE	CAN SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
25	32	settable	TIM1_UP	TIM1 Update interrupt	0x0000_00A4
26	33	settable	TIM1_TRG_COM	TIM1 Trigger and Commutation interrupts	0x0000_00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
31	38	settable	I2C1_EV	I ² C1 event interrupt	0x0000_00BC
32	39	settable	I2C1_ER	I ² C1 error interrupt	0x0000_00C0
33	40	settable	I2C2_EV	I ² C2 event interrupt	0x0000_00C4
34	41	settable	I2C2_ER	I ² C2 error interrupt	0x0000_00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000_00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000_00D0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8
39	46	settable	USART3	USART3 global interrupt	0x0000_00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0
41	48	settable	RTCAlarm	RTC alarm through EXTI line interrupt	0x0000_00E4

Table 63. Vector table for other STM32F10xxx devices (continued)

Position	Priority	Type of priority	Acronym	Description	Address
42	49	settable	USBWakeup	USB wakeup from suspend through EXTI line interrupt	0x0000_00E8
43	50	settable	TIM8_BRK	TIM8 Break interrupt	0x0000_00EC
44	51	settable	TIM8_UP	TIM8 Update interrupt	0x0000_00F0
45	52	settable	TIM8_TRG_COM	TIM8 Trigger and Commutation interrupts	0x0000_00F4
46	53	settable	TIM8_CC	TIM8 Capture Compare interrupt	0x0000_00F8
47	54	settable	ADC3	ADC3 global interrupt	0x0000_00FC
48	55	settable	FSMC	FSMC global interrupt	0x0000_0100
49	56	settable	SDIO	SDIO global interrupt	0x0000_0104
50	57	settable	TIM5	TIM5 global interrupt	0x0000_0108
51	58	settable	SPI3	SPI3 global interrupt	0x0000_010C
52	59	settable	UART4	UART4 global interrupt	0x0000_0110
53	60	settable	UART5	UART5 global interrupt	0x0000_0114
54	61	settable	TIM6	TIM6 global interrupt	0x0000_0118
55	62	settable	TIM7	TIM7 global interrupt	0x0000_011C
56	63	settable	DMA2_Channel1	DMA2 Channel1 global interrupt	0x0000_0120
57	64	settable	DMA2_Channel2	DMA2 Channel2 global interrupt	0x0000_0124
58	65	settable	DMA2_Channel3	DMA2 Channel3 global interrupt	0x0000_0128
59	66	settable	DMA2_Channel4_5	DMA2 Channel4 and DMA2 Channel5 global interrupts	0x0000_012C

Microcontroller



External interrupt/event controller (EXTI)

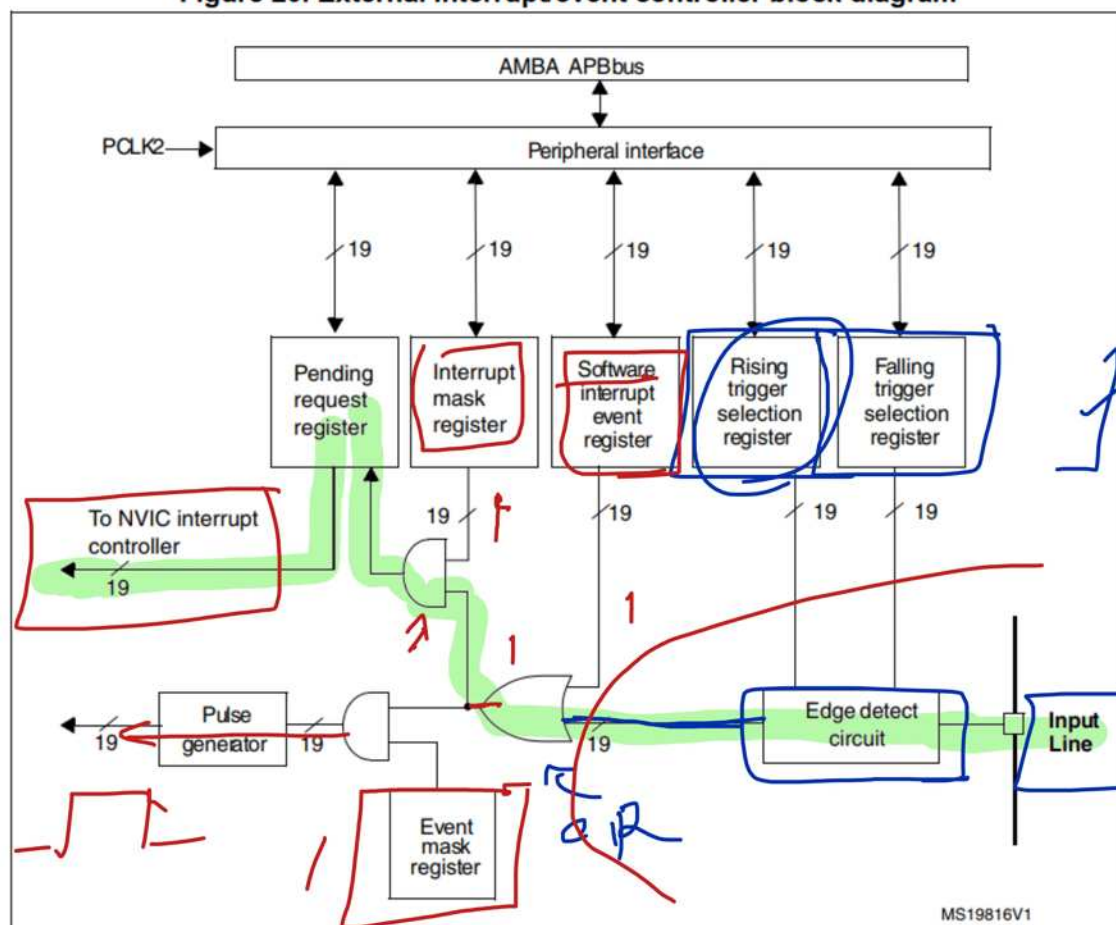
The external interrupt/event controller consists of up to 20 edge detectors in connectivity line devices, or 19 edge detectors in other devices for generating event/interrupt requests. Each input line can be independently configured to select the type (event or interrupt) and the corresponding trigger event (rising or falling or both). Each line can also be masked independently. A pending register maintains the status line of the interrupt requests

Main features

The EXTI controller main features are the following:

- Independent trigger and mask on each interrupt/event line
- Dedicated status bit for each interrupt line
- Generation of up to 20 software event/interrupt requests
- Detection of external signal with pulse width lower than APB2 clock period. Refer to the electrical characteristics section of the datasheet for details on this parameter.

Figure 20. External interrupt/event controller block diagram



Wakeup event management

The STM32F10xxx is able to handle external or internal events in order to wake up the core (WFE). The wakeup event can be generated either by:

- enabling an interrupt in the peripheral control register but not in the NVIC, and enabling the **SEVONPEND** bit in the Cortex®-M3 System Control register. When the MCU resumes from WFE, the peripheral interrupt pending bit and the peripheral NVIC IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared.
- or configuring an external or internal EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bit corresponding to the event line is not set.

In connectivity line devices, Ethernet wakeup events also have the WFE wakeup capability.

Functional description

To generate the interrupt, the interrupt line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the interrupt request by writing a '1' to the corresponding bit in the interrupt mask register. When the selected edge occurs on the external interrupt line, an interrupt request is generated. The pending bit corresponding to the interrupt line is also set. This request is reset by writing a '1' in the pending register.

To generate the event, the event line should be configured and enabled. This is done by programming the two trigger registers with the desired edge detection and by enabling the event request by writing a '1' to the corresponding bit in the event mask register. When the selected edge occurs on the event line, an event pulse is generated. The pending bit corresponding to the event line is not set.

An interrupt/event request can also be generated by software by writing a '1' in the software interrupt/event register.

Hardware interrupt selection

To configure the 20 lines as interrupt sources, use the following procedure:

- Configure the mask bits of the 20 Interrupt lines (EXTI_IMR)
- Configure the Trigger Selection bits of the Interrupt lines (EXTI_RTSR and EXTI_FTSR)
- Configure the enable and mask bits that control the NVIC IRQ channel mapped to the External Interrupt Controller (EXTI) so that an interrupt coming from one of the 20 lines can be correctly acknowledged.

Hardware event selection

To configure the 20 lines as event sources, use the following procedure:

- Configure the mask bits of the 20 Event lines (EXTI_EMR)
- Configure the Trigger Selection bits of the Event lines (EXTI_RTSR and EXTI_FTSR)

Software interrupt/event selection

The 20 lines can be configured as software interrupt/event lines. The following is the procedure to generate a software interrupt.

- Configure the mask bits of the 20 Interrupt/Event lines (EXTI_IMR, EXTI_EMR)
- Set the required bit of the software interrupt register (EXTI_SWIER)

External interrupt/event line mapping

The 112 GPIOs are connected to the 16 external interrupt/event lines in the following manner:



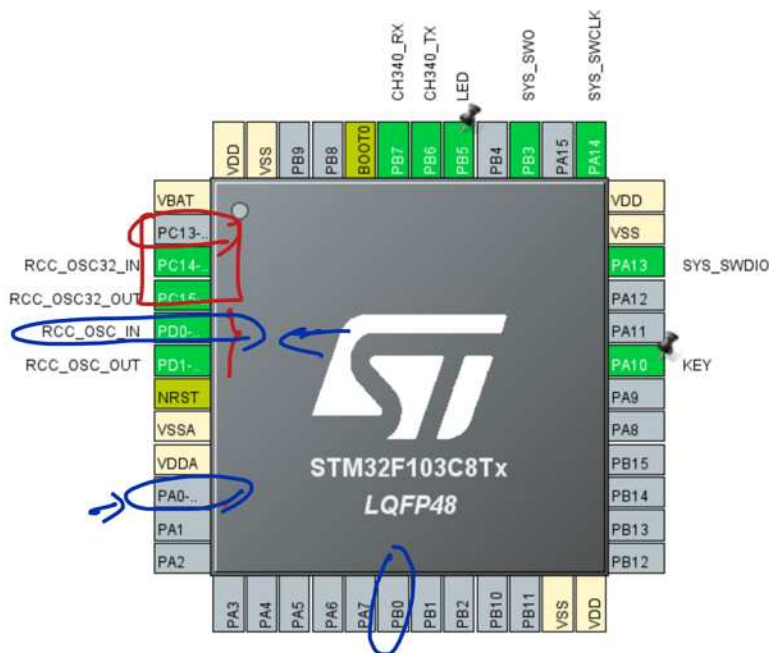
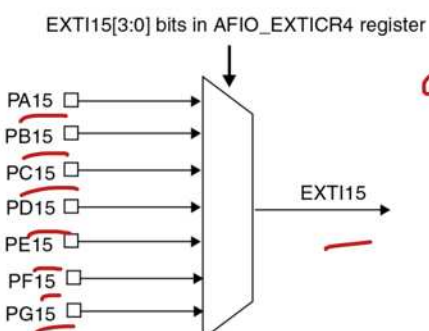
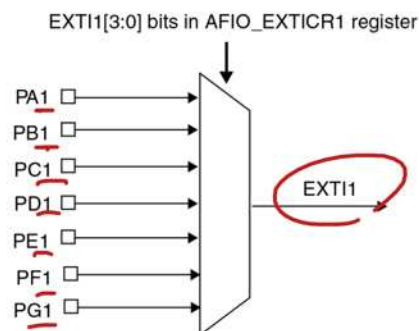
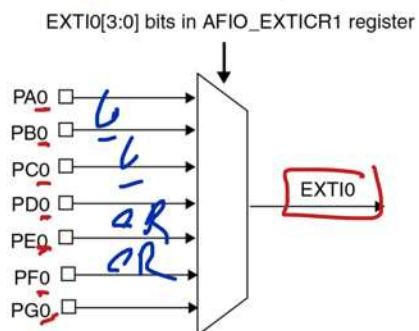
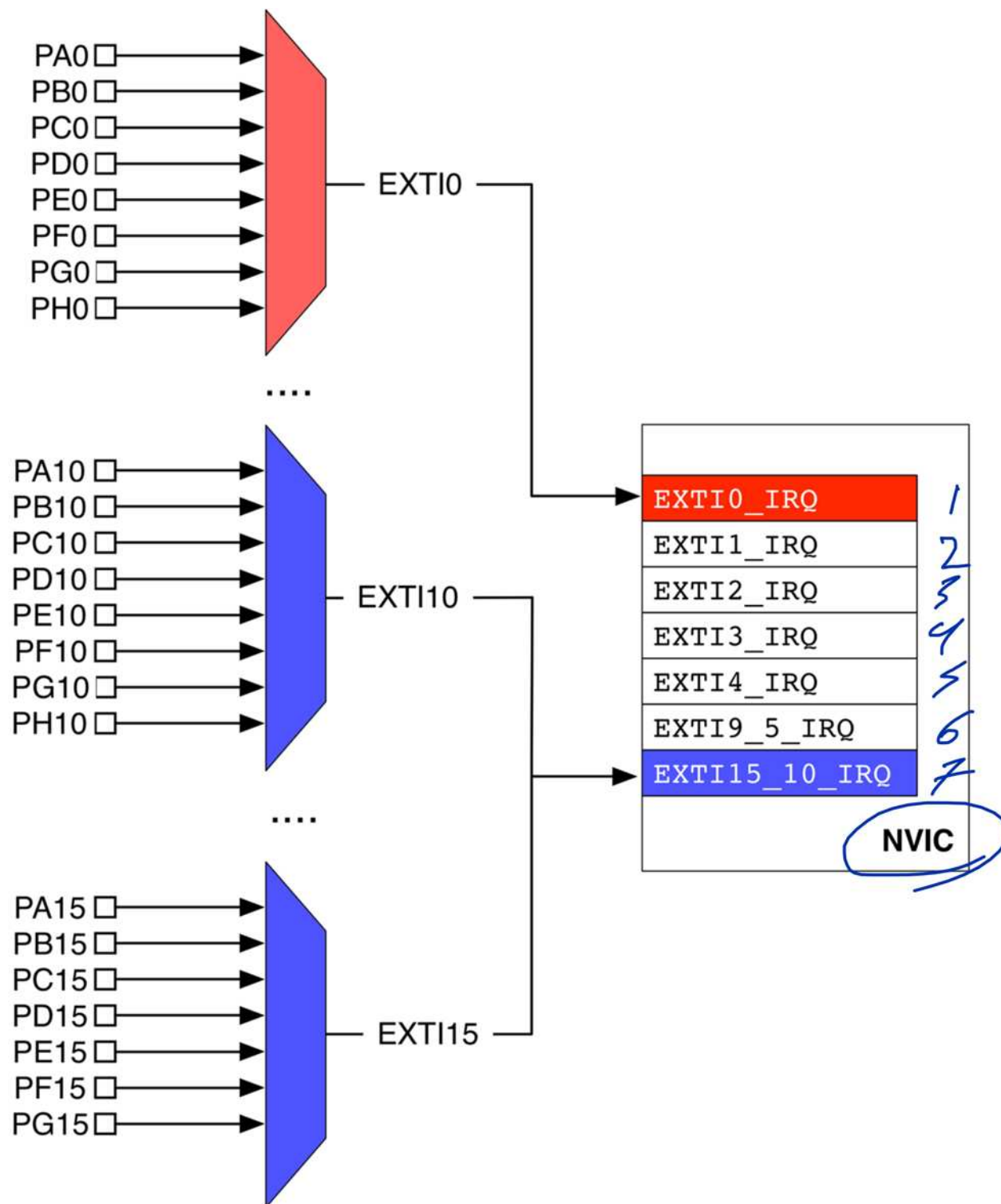


Figure 21. External interrupt/event GPIO mapping



0 - 15 -> 16



Interrupt priority mechanism in Cortex-M3/4/7/33 is more advanced than the one available in Cortex-M0/0+ based microcontrollers. Developers have a higher degree of flexibility, and this is often source of several headaches for novices. Moreover, the way interrupt priority is presented both in the ARM and ST documentation is a little bit counterintuitive.

In Cortex-M3/4/7/33 cores the priority of each interrupt is defined through the IPR register. This is an 8bit register in the ARMv7-M core architecture that allows up to 255 different priority levels. However, in practice, STM32 MCUs implementing Cortex-M3/4/7 cores use only the four upper bits of this register, while STM32 MCUs based on Cortex-M33 cores use only three upper bits.



Figure 7.15: The content of IPR register on an STM32 MCU based on Cortex-M3/4/7 core

4bit
2⁴ = 16

15 → 15

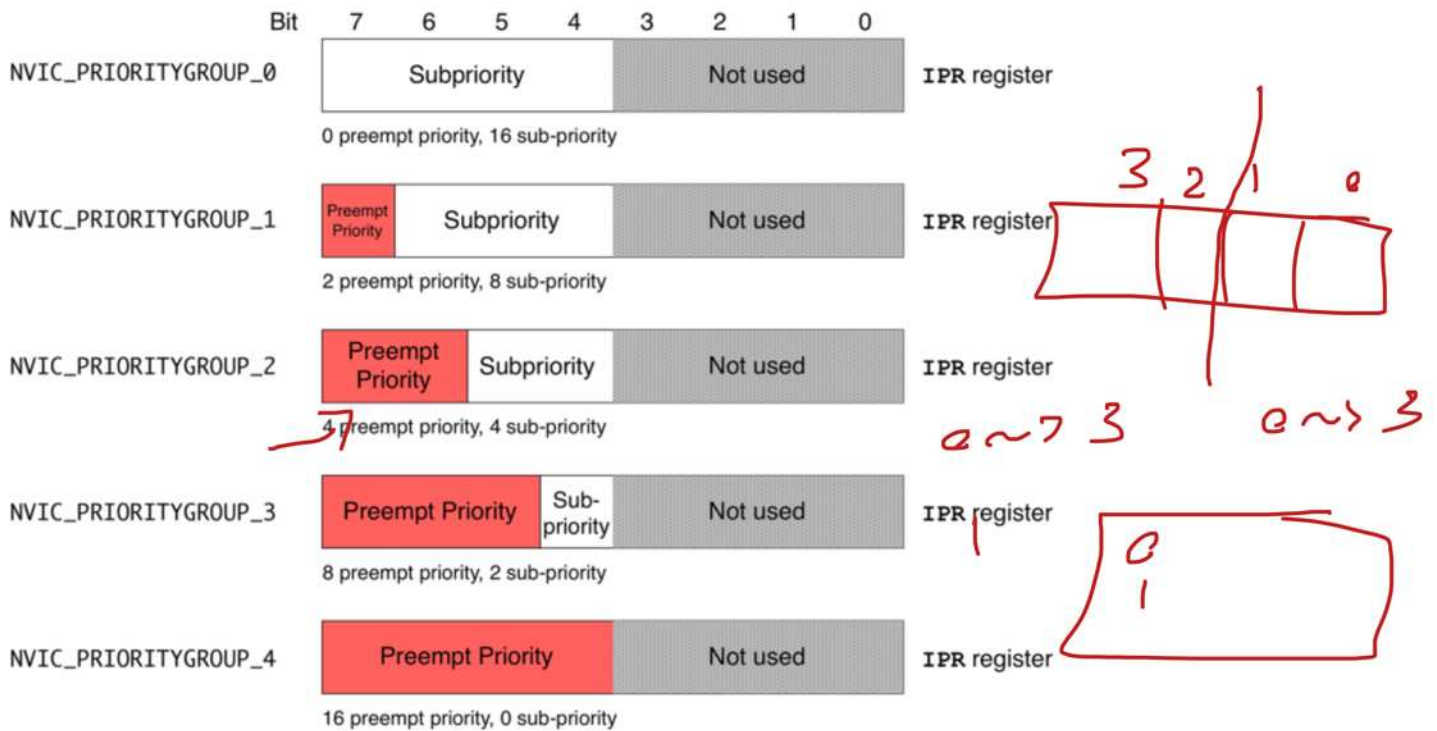


Figure 7.18: The subdivision of IPR bits between preemption priority and sub-priority

Table 2: The number of preemption priority level available based on the current *priority grouping* schema

NVIC Priority Group	Number of preemption priority levels	Number of sub-priority levels
NVIC_PRIORITYGROUP_0	0	16
NVIC_PRIORITYGROUP_1	2	8
NVIC_PRIORITYGROUP_2	4	4
NVIC_PRIORITYGROUP_3	8	2
NVIC_PRIORITYGROUP_4	16	0

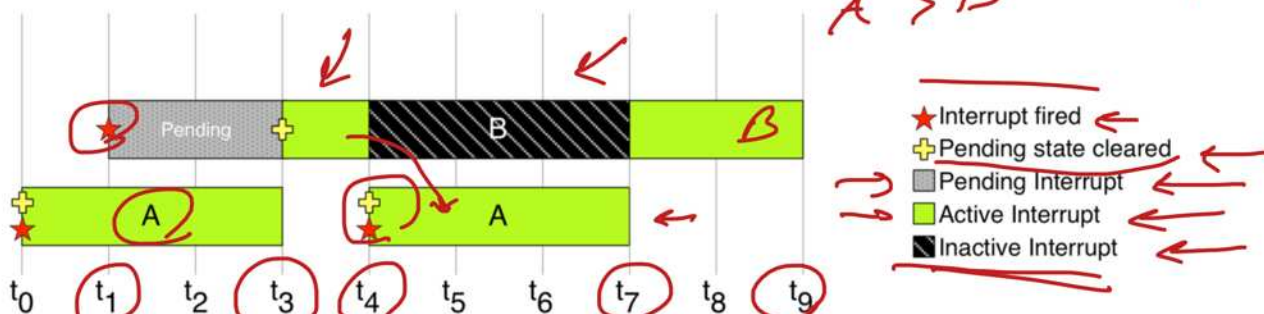


Figure 7.8: The relation between the active status and interrupts priority

- Reset:** this exception is raised just after the CPU resets. Its handler is the real entry point of the running firmware. In an STM32 application all starts from this exception. The handler contains some assembly-coded functions designed to initialize the execution environment, such as the main stack, the `.bss` area, etc. [Chapter 22](#) dedicated to the booting process will explain in depth.
- NMI:** this is a special exception, which has the highest priority after the *Reset* one. Like the *Reset* exception, it cannot be masked, and it can be associated to critical and non-deferrable activities. In all STM32 microcontrollers it is linked to the *Clock Security System (CSS)*. CSS is a self-diagnostic peripheral that detects the failure of the external clock, called HSE. If this happens, HSE is automatically disabled (this means that the internal HSI is automatically enabled) and an NMI interrupt is raised to inform the software that something is wrong with the HSE. More about this feature in [Chapter 10](#).
- Hard Fault:** is the generic fault exception, and hence related to software interrupts. When the other fault exceptions are disabled, it acts as a collector for all types of exceptions (e.g., a memory access to an invalid location raised the Hard Fault exceptions if the Bus Fault one is not enabled).

- **Memory Management Fault¹**: it occurs when executing code attempts to access an illegal location or violates a rule of the Memory Protection Unit (MPU). More about this in [Chapter 20](#).
- **Bus Fault¹**: it occurs when AHB interface receives an error response from a bus slave (also called *prefetch abort* if it is an instruction fetch, or *data abort* if it is a data access). Can also be caused by other illegal accesses (e.g., an access to a non-existent SRAM memory location).
- **Usage Fault¹**: it occurs when there is a program error such as an illegal instruction, alignment problem, or attempt to access a non-existent co-processor.
- **SVCCall**: this is not a fault condition, and it is raised when the *Supervisor Call* (SVC) instructions is called. This is used by *Real Time Operating Systems* to execute instructions in privileged state (a task needing to execute privileged operations executes the SVC instruction, and the OS performs the requested operations - this is the same behavior of a system call in other OSes).
- **Debug Monitor¹**: this exception is raised when a software debug event occurs while the processor core is in Monitor Debug-Mode. It is also used as exception for debug events like breakpoints and watchpoints when software-based debug solution is used.
- **PendSV**: this is another exception related to RTOS. Unlike the SVC call exception, which is executed immediately after an SVC instruction is executed, the PendSV can be delayed. This allows the RTOS to complete tasks with higher priorities.
- **SysTick**: this exception is also usually related to RTOS activities. Every RTOS needs a timer to periodically interrupt the execution of current code and to switch to another task. All STM32 microcontrollers provide a *SysTick* timer, internal to the Cortex-M core. Even if every other timer may be used to schedule system activities, the presence of a dedicated timer ensures portability among all STM32 families (due to optimization reasons related to the internal die of the MCU, not all timers could be available as external peripheral). Moreover, even if we aren't using an RTOS in our firmware, it is important to keep in mind that the ST CubeHAL uses the *SysTick* timer to perform internal time-related activities (and it assumes that the *SysTick* timer is configured to generate an interrupt every 1ms).

Number	Exception type	Priority ^a	Function
1	Reset	-3	Reset
2	NMI	-2	Non-Maskable Interrupt
3	Hard Fault	-1	All classes of Fault, when the fault cannot activate because of priority or the Configurable Fault handler has been disabled.
4	Memory Management ^c	Configurable ^b	MPU mismatch, including access violation and no match. This is used even if the MPU is disabled or not present.
5	Bus Fault ^c	Configurable	Pre-fetch fault, memory access fault, and other address/memory related.
6	Usage Fault ^c	Configurable	Usage fault, such as Undefined instruction executed or illegal state transition attempt.
7	SecureFault ^d	Configurable	SecureFault is available when the CPU runs in <i>Secure state</i> . It is triggered by the various security checks that are performed. For example, when jumping from Non-secure code to an address in Secure code that is not marked as a valid entry point.
8-10	-	-	RESERVED
11	SVCCall	Configurable	System service call with SVC instruction.
12	Debug Monitor ^c	Configurable	Debug monitor – for software based debug.
13	-	-	RESERVED
14	PendSV	Configurable	Pending request for system service.
15	SysTick	Configurable	System tick timer has fired.
16- [47/239/479] ^e	IRQ	Configurable	IRQ Input